MémoriA — Fiche de révision QCM

Un bloc par thème de la grille. Apprendre les **mots en gras**, puis faire le quiz à la fin.

1. Architecture

- **Django = MVT** : **M**odèle (données, `models.py`) → **V**ue (logique, `views.py`) → **T**emplate (HTML).
- Trajet d'une requête : URL → `urls.py` → vue → **ORM** (base) → template → page HTML.
- 1 projet `MemoriA` + **6 apps** : Account (commun), Journal, Voice, Face, Memory, Reminder.
- Dans chaque app : `services.py` = **appels IA**, `tasks.py` = **tâches Celery**, `tests/`, `evaluation/`.
- **Vues génériques** : `ListView` (liste), `DetailView` (détail), `CreateView`, `UpdateView`, `DeleteView` (= CRUD).
- **ORM** : on écrit du Python (`JournalEntry.objects.filter(user=u)`) au lieu du SQL.
- Base : **SQLite** (un fichier). Front : **Bootstrap 5** + JavaScript simple.

2. Asynchrone (Celery + Redis)

- Problème : une IA prend 5 à 60 s → la page serait **bloquée**.
- Solution :

1. La vue lance `tache.delay()` et **répond tout de suite**.
2. **Redis** = la **file d'attente** (garde la tâche et le résultat).
3. Le **worker Celery** exécute la tâche.
4. La page demande toutes les **2 s** à `ai/task/<id>/` : `PENDING` / `SUCCESS` / `FAILURE`.

- Pendant l'attente : **indicateur de chargement** (`aria-live`). Erreur : **message + bouton Réessayer**.
- **Celery Beat** = réveil : envoie les rappels toutes les 5 min.
- En test : `CELERY_TASK_ALWAYS_EAGER = True` (exécuté tout de suite) et **IA simulées (mock)**.

3. Les IA par module

Module	IA	Modèle	Métrique (résultat)
Journal	Recherche **par le sens** + 3 souvenirs similaires	**MiniLM** (sentence-transformers) → **embeddings** + **cosinus**	**MRR 0,875**
	Suggestion d'étiquettes	**mDeBERTa zero-shot**	**F1 0,75**
Voice	Transcription de la voix	**Whisper**	**WER 5,9 %**
	Émotion + alerte 3 jours négatifs	**xlm-emo-t**	92 % sur 4 émotions
Face	Légende + objets	**BLIP** + **DETR**	CLIP, précision objets 0,875
	Détection des visages	**OpenCV YuNet**	Recall 1,0
	Reconnaissance + regroupement	**OpenCV SFace** + **DBSCAN**	Accuracy 100 %
Memory	Récit de la journée (multimodal)	**Gemini 2.5 Flash**	Grille qualitative /5
	Illustration	**Stable Diffusion** (SDXL / SD-Turbo)	**CLIP 31,6**
	Bilan semaine/mois	**KeyBERT**	Precision@5 0,5
Reminder	Meilleure heure	**KMeans** (scikit-learn)	**MAE 2,5 min**
	Question personnalisée	**CamemBERT-NER**	**F1 0,80**

Mots à connaître

- **Embedding** : un texte transformé en vecteur de nombres. Même sens → vecteurs proches.

- **Similarité cosinus** : 1 = même sens, 0 = aucun rapport.
- **Zero-shot** : classer dans des catégories **jamais apprises**.
- **NER** : trouver les noms propres (personnes, lieux, organisations).
- **KMeans** : regroupe des points en paquets ; **DBSCAN** : pareil sans connaître le nombre de groupes.
- **Multimodal** : texte + images en même temps.
- **Moteur de secours** : si l'API Hugging Face échoue → modèle **local** (`*_BACKEND=auto`).

Métriques

- **Precision** = parmi ce que l'IA propose, ce qui est **juste**.
- **Recall** = parmi ce qu'il fallait trouver, ce qui est **trouvé**.
- **F1** = équilibre entre Precision et Recall.
- **Matrice de confusion** = tableau attendu / prédit ; la **diagonale** = bonnes réponses.
- **Accuracy** = bonnes réponses / total.
- **MRR** = moyenne de 1/rang de la 1re bonne réponse (1 = toujours en tête).
- **WER** = taux de mots faux dans une transcription (**plus bas = mieux**).
- **MAE** = erreur moyenne (ici en minutes).

Règles IA du projet

- Appels IA **uniquement dans** `services.py`, dans un `try/except`.
- Chaque appel est **mesuré** (`ActivityLog` : service, statut, **durée en ms**) → page **Statistiques**.
- Modèles chargés **une seule fois** (cache `lru_cache`), embeddings gardés en base.
- Clés API dans `.env`, jamais dans le code.

4. Sécurité (OWASP)

Risque	Protection
CSRF (formulaire envoyé par un autre site)	`{% csrf_token %}` dans chaque formulaire
Injection SQL	**ORM** uniquement, pas de SQL brut
XSS (script injecté)	Templates qui **échappent** le HTML
Mots de passe	**Hashés** par Django (PBKDF2), jamais en clair
Secrets	`.env` (dans `.gitignore`) : `SECRET_KEY`, `DEBUG`, clés API
Production	`DEBUG=False`, HTTPS forcé (`SECURE_SSL_REDIRECT`, cookies `SECURE`, HSTS), `check --deploy` OK
Fichiers envoyés	Type vérifié par les **premiers octets**, taille et durée max

Rôles et permissions

- `LoginRequiredMixin` → pas connecté = **302** vers la connexion.
- `PermissionRequiredMixin` → page Statistiques réservée à l'admin, sinon **403**.
- Groupes **Utilisateur** et **Administrateur**.
- `get_queryset()` filtre sur `request.user` → données d'un autre = **404**.
- `user` rempli dans `form_valid()`, jamais par le formulaire.
- Pages **403, 404, 500** personnalisées.
- Validation **serveur** (`clean()`) **+ client** (`required`, `maxlength`) — seul le serveur est fiable.

5. Tests

- **Unitaires** : un petit morceau (modèle, `clean()`, `__str__`, `unique_together`).
- **Intégration** : une vue complète avec `django.test.Client` (codes 200 / 302 / 403 / 404).

- **Mock** : fausse réponse de l'IA → on teste **succès ET échec** sans Internet.
- **547 tests**, **couverture 95 %** : `coverage run manage.py test` puis `coverage report`.
- **Couverture** = % des lignes de code exécutées par les tests.

6. Accessibilité

- Balises : `<header>`, `<nav>`, `<main>`, `<footer>`, **un seul** `<h1>`, titres dans l'ordre.
- `alt` sur les images, `aria-label` sur les boutons-icônes, `aria-live` sur le chargement, `<label>` sur chaque champ.
- Contraste **WCAG AA ≥ 4,5:1**, police **≥ 16 px**.
- Clavier : lien **« Aller au contenu »**, **focus visible**, tout faisable avec Tab / Entrée.
- `lang="fr"`, `<title>` différent sur chaque page. Audit : **Lighthouse**, **axe**.

7. Intégration et déploiement

- **Git** : chacun commite son module (sinon note individuelle / 2).
- **CI GitHub Actions** : à chaque push → installation, `migrate`, `check`, tests + coverage.
- **Docker** : `docker compose up` lance `web` + `worker` + `beat` + `redis`.
- **gunicorn** = serveur web de production (`runserver` = développement seulement).
- **whitenoise** = sert le CSS/JS en production.
- **Render** (gratuit) : 512 Mo de mémoire → IA par API seulement, disque effacé à chaque redémarrage, veille après 15 min.

Quiz express (cacher la réponse et tester)

1. Que signifie MVT ? → **Modèle, Vue, Template.**
2. Où sont les appels IA ? → **Dans `services.py`.**
3. Pourquoi Celery ? → **Ne pas bloquer la requête HTTP pendant l'IA.**
4. Rôle de Redis ? → **File d'attente des tâches et des résultats.**
5. Un utilisateur ouvre l'entrée d'un autre par l'URL ? → **404** (filtre `request.user`).
6. Un utilisateur non connecté ouvre une page privée ? → **302** vers la connexion.
7. Un simple utilisateur ouvre les Statistiques ? → **403.**
8. Protection contre l'injection SQL ? → **L'ORM.**
9. Protection contre le CSRF ? → **Le jeton `{% csrf_token %}`.**
10. Où est la `SECRET_KEY` ? → **Dans `.env`**, jamais dans Git.
11. Precision vs Recall ? → **Precision = justesse de ce qui est proposé ; Recall = part de ce qui est trouvé.**
12. Que montre la diagonale d'une matrice de confusion ? → **Les bonnes réponses.**
13. Modèle de transcription ? → **Whisper** ; sa métrique ? → **WER.**
14. Qu'est-ce qu'un embedding ? → **Un texte transformé en vecteur de nombres.**
15. Pourquoi le zero-shot pour les étiquettes ? → **Chaque utilisateur crée ses propres étiquettes, pas d'entraînement possible.**
16. Modèle des récits ? → **Gemini 2.5 Flash.** Des images ? → **Stable Diffusion.**
17. Pourquoi les heures sur un cercle (KMeans) ? → **Pour que 23 h 55 et 0 h 05 soient proches.**
18. Que mesure la couverture ? → **Le % de code exécuté par les tests** (ici 95 %).

19. Rôle de gunicorn ? → **Serveur web de production.**
20. Pourquoi `DEBUG=False` en production ? → **Ne pas montrer le code et les erreurs techniques.**